



**FACULTAD
DE INGENIERIA**

Universidad de Buenos Aires

[75.12 / 95.04] INTRODUCCIÓN A LOS SISTEMAS
DISTRIBUIDOS (75.43)

TP N1: FILE TRANSFER

1^{ER} CUATRIMESTRE 2026

AUTORES

Fabrizio Batastini - 111828
Facundo Madotta - 112180
Manuel Peñalva - 111696
Mateo Nowenstein - 112063
Joaquin Schapira - 112459

FECHA DE ENTREGA

Jueves 08 de Mayo de 2026

CALIFICACIÓN

GRUPO

6

CURSO

02

Índice

1. Introducción	2
2. Hipótesis y suposiciones realizadas	2
3. Implementación	4
3.1. Arquitectura general:	4
3.2. Arquitectura de hilos y sockets:	5
3.3. Análisis de Puertos Efímeros	6
3.4. Estructura del paquete:	6
3.5. Inicio de Comunicación:	7
3.6. Mecanismos de recuperación ante errores	8
3.7. Stop-And-Wait	8
3.7.1. Manejo de errores	8
3.7.2. Ventajas y desventajas	8
3.7.3. Flujo de envío (Sender)	9
3.7.4. Flujo de recepción (Receiver)	9
3.8. Go-Back-N	10
3.8.1. Manejo de irregularidades en la red	10
3.9. Manejo de errores de aplicación:	10
3.10. Fin de Comunicación	11
4. Pruebas	11
4.0.1. Resultados experimentales	11
4.0.2. Archivo 1MB	12
4.0.3. Archivo 5MB:	15
4.1. Análisis de Throughput (rendimiento)	19
4.2. Análisis de Protocolos con Wireshark	20
4.2.1. Pérdida de paquetes con Stop and Wait	20
4.3. Pérdida de paquetes y manejo de ventana de envío bajo Go-Back-N	21
5. Preguntas teóricas	22
5.1. Arquitectura Cliente-Servidor	23
5.2. ¿Cuál es la función de un protocolo de capa de aplicación?	23
5.3. Detalle del protocolo de aplicación desarrollado	23
5.4. TCP vs UDP	24
5.5. Control de congestión	24
5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo?	25
6. Dificultades encontradas	25
7. Conclusión	26

1. Introducción

En este trabajo práctico se va a desarrollar una aplicación de transferencia de archivos basada en una arquitectura cliente-servidor, implementada sobre la capa de transporte utilizando el protocolo UDP. A diferencia de TCP, UDP no ofrece mecanismos de confiabilidad, orden ni control de flujo, lo que implica la necesidad de implementar dichos mecanismos a nivel de aplicación.

En base a este objetivo, se diseñan e implementan dos protocolos de transferencia confiable de datos: *Stop-and-Wait* y *Go-Back-N*. Los dos permiten garantizar la correcta recepción de los datos mediante el uso de confirmaciones (ACKs) y retransmisiones, aunque varían en términos de eficiencia y canales utilizados.

La aplicación soporta dos operaciones principales: *Upload*, que permite transferir archivos desde el cliente hacia el servidor, y *Download*, que realiza la transferencia de archivos del servidor hacia el cliente. Estas operaciones se ejecutan sobre una red simulada usando *Mininet*, lo que permite introducir condiciones específicas de pérdida de paquetes y evaluar el comportamiento de los protocolos implementados.

El objetivo del trabajo es analizar y comparar el desempeño de los protocolos *Stop-and-Wait* y *Go-Back-N* en la transferencia de archivos, evaluando su cómo funcionan en el caso de pérdidas de paquetes y su impacto en el tiempo de transferencia.

2. Hipótesis y suposiciones realizadas

■ Diferencia de performance entre protocolos:

Se plantea como hipótesis que el protocolo *Go-Back-N* presentará un menor tiempo de transferencia que *Stop-and-Wait* en escenarios sin pérdida de paquetes o con baja tasa de error. Esto se puede deber a que *Go-Back-N* permite el envío de más de un paquete sin esperar la confirmación individual de cada uno, aprovechando eficientemente el canal de comunicación mediante el uso de una ventana deslizante, a diferencia de *Stop-and-Wait* que espera el ACK de cada paquete antes de hacer el próximo envío.

■ Uso de timeouts y retransmisiones:

Se asume que el uso de *timeouts* ayuda detectar pérdidas de paquetes de manera eficiente. Esto permite evitar tanto retransmisiones innecesarias como períodos de inactividad prolongados. Un timeout demasiado corto podría llevar a retransmisiones repetidas, mientras que uno más largo hace que el tiempo total de transferencia suba. Esto hace que lo que terminemos buscando es un punto medio donde se maximice el rendimiento en general del protocolo.

Para establecer los tiempos de *timeouts* tomamos:

- Tamaño paquete = 1024B

- Cantidad Paquetes = 5 MB / 1024 B = 4882
- RTT sin pérdidas = Tiempo de envío / Cant Paquetes

Con los tiempos de envío testeados, el más grande que se obtuvo en un archivo de 5MB fue de 1,8 seg.

- $RTT = 1,8 / 4882 = 0,0003$ seg
- Se decidió utilizar un timeout considerablemente mayor al RTT estimado para evitar retransmisiones innecesarias.
- Timeout $\approx 0,05$ s (50 ms)

Si bien el RTT teórico de la red local es muy bajo, de 0,3 ms, un timeout tan corto conlleva a retransmisiones innecesarias, ocupando ancho de banda excesivamente. Además, poner un timeout tan bajo no permitiría tardanza en el procesamiento de parte de la aplicación del receptor, provocando retransmisiones cuya causa no es la pérdida de paquetes o la congestión. Por eso se optó por aumentarlo a 50 ms, la cual es una ventana de tiempo que permite recuperar los paquetes perdidos rápidamente.

■ **Concurrencia:**

Se asume un modelo de concurrencia para permitirle al servidor interactuar con distintos clientes que quieran hacer transferencias al mismo tiempo, esto sin que una transferencia bloquee a las demás. De esta manera se pensó que cada cliente tenga su propio thread, lo que favorece la organización y el manejo independiente de cada conexión. Aparte esperamos que mejoren los tiempos de respuesta en el caso de haber distintos clientes activos al mismo tiempo para que no colapse el servidor manejando distintas conexiones continuas.

■ **Handshake:**

Entendemos que necesitamos una sólida implementación del mecanismo de handshake entre el cliente y el servidor al inicio de la transferencia. Por lo que se ve necesario una implementación de un *3-Way-Handshake* al inicio de la transferencia. Ya que mal implementado podría generar muchos problemas al futuro, inconsistencias en el estado de comunicación como pérdida de mensajes iniciales o una desincronización entre cliente y servidor. De esta manera, se garantiza que las operaciones de *upload* y *download* se realicen correctamente.

■ **Checksum:**

No implementamos ningún algoritmo de checksum a nivel de aplicación para validar la integridad de los datos. Se asume como hipótesis que el mecanismo nativo de checksum de 16 bits incluido en el header del protocolo UDP es suficiente para detectar la corrupción de bits en tránsito. Por lo tanto, asumimos que si un datagrama sufre alteraciones, el SO lo descartará antes de entregarlo al socket. Nuestra implementación solamente se centrará en resolver la pérdida y el desorden de los paquetes, pero no la corrupción de los mismos.

- **Arquitectura y orden de Bytes:**

Dado que el cliente y el servidor podrían ejecutarse en máquinas con arquitecturas distintas, asumimos como regla de diseño el uso de *Big-Endian* para la serialización de los encabezados de nuestros paquetes. Esto garantiza que las variables de múltiples bytes sean interpretadas idénticamente en ambos extremos de la conexión.

- **Múltiples puertos disponibles:**

Al tener que conectar múltiples clientes a un único servidor, implementamos que cada cliente tenga un puerto disponible y un socket para sí mismo; por lo que asumimos que habrá múltiples puertos para los clientes que necesitan ser conectados.

- **Ventana fija en Go-Back-N:**

Para la implementación del protocolo *Go-Back-N*, se asume que el tamaño de la ventana de transmisión *N* es estático (como lo es teóricamente) y no variará dinámicamente según la congestión de la red durante el ciclo de vida de la transferencia.

- **Tamaño del paquete:**

Se fijó el tamaño máximo del datagrama de la aplicación en 1024 bytes como una decisión de diseño basada en el MTU estándar de las redes. Como nuestro paquete encapsula 1024 bytes + los del encabezado de UDP e IPv4, el tamaño total en transmisión es aproximadamente 1050 bytes. Por lo que, dejando nuestro paquete en 1024 garantiza un margen de seguridad para que el datagrama viaje sin sufrir fragmentación. Además, se debe evitar la fragmentación ya que UDP descarta todo el paquete si no llega completo.

3. Implementación

3.1. Arquitectura general:

La implementación del sistema se basa en una arquitectura cliente-servidor donde se utiliza UDP como protocolo de la capa de transporte. Dado que UDP no está orientado a conexión y carece de mecanismos de gestión, fue necesario diseñar un mecanismo que permita organizar a los distintos clientes y cómo interactúan con el servidor.

Para lograr esto, el servidor principal cuenta con un encargado de recibir mensajes de distintos clientes entrantes denominado *Client Acceptor*. El cual se encarga de detectar nuevos usuarios y derivarlos a *Client Handlers* específicos para ejecutar las operaciones. Cada cliente es atendido por un thread distinto asociado a su propio *Client Handler* independiente, lo que permite escalar el servidor y manejar múltiples transferencias simultáneas sin cuello de botella.

Una vez instanciado, el *Client Handler* es el responsable de procesar pedidos de su cliente asignado. Interpreta la operación solicitada como (*Upload* y *Download*) aparte del protocolo específico como (*Stop-And-Wait* o *Go-Back-N*). En función de lo pedido se especifica cómo se hace la transferencia de datos específica.

3.2. Arquitectura de hilos y sockets:

Para optimizar el rendimiento, la ejecución de la instancia del protocolo se divide en dos hilos independientes:

- **Hilo Sender:** Dedicado exclusivamente a despachar los paquetes de datos hacia la red
- **Hilo Receiver:** Encargado de escuchar de forma asíncrona en el socket a la espera de los paquetes de confirmación (ACKs) o datos entrantes.

Gracias a esta separación, evita que las operaciones de lectura bloqueantes del socket detengan el envío continuo de datos, permitiendo un flujo bidireccional y mejorando la eficiencia.

A nivel código, se arman los sockets UDP usando `AF_INET` y `SOCK_DGRAM` de la biblioteca de python. El Client Acceptor le hace un `bind()` al puerto público para quedarse escuchando únicamente los mensajes iniciales de conexión con un `recvfrom()` (SYN del cliente).

Cuando llega un SYN, el Acceptor crea un Client Handler independiente por cada cliente. El mismo arma su propio socket UDP nuevo para ese cliente, por lo que hace `bind(('0.0.0.0', 0))`; haciendo que el sistema operativo le asigne un puerto efímero libre al azar. Desde este puerto nuevo, el Handler mediante `sendto()` le responde el ACK al cliente, quien al recibirlo, actualiza su destino y manda el resto de la transferencia hacia este nuevo puerto. De esta manera, se aísla el tráfico de cada cliente en su propio socket, liberando el puerto principal al instante.

Por último, para evitar que los hilos se bloqueen permanentemente esperando recibir algo en `recvfrom()`, se le configura un temporizador a los sockets con `settimeout()` para que luego se le controle la excepción `socket.timeout` y poder manejar la pérdida de paquetes.

Para examinar este comportamiento, analizaremos la salida de la terminal mediante el comando `netstat`, observando cómo se asignan los nuevos puertos efímeros cada vez que se conecta un cliente. Además, notaremos que la columna de estado permanece vacía; esto se debe a que la transferencia se realiza sobre UDP (un protocolo no orientado a conexión), por lo que, a diferencia de TCP, el sistema operativo no registra estados formales como `ESTABLISHED`, `TIME_WAIT` o `LISTEN`.

3.3. Análisis de Puertos Efímeros

```

facu@facu-facu:~/Escritorio/redes/tp1-redes$ sudo python3 topology_mininet.py
mininet> xterm h1 h2
mininet> h1 netstat -una
Conexiones activas de Internet (servidores y establecidos)
Proto Recib Enviad Dirección local      Dirección remota      Estado
mininet> h1 netstat -una
Conexiones activas de Internet (servidores y establecidos)
Proto Recib Enviad Dirección local      Dirección remota      Estado
udp      0      0 10.0.0.1:8080        0.0.0.0:*
mininet> h1 netstat -una
Conexiones activas de Internet (servidores y establecidos)
Proto Recib Enviad Dirección local      Dirección remota      Estado
udp      0      0 0.0.0.0:35331        0.0.0.0:*
udp      0      0 10.0.0.1:8080        0.0.0.0:*
mininet> h1 netstat -una
Conexiones activas de Internet (servidores y establecidos)
Proto Recib Enviad Dirección local      Dirección remota      Estado
udp      0      0 10.0.0.1:8080        0.0.0.0:*
mininet> █

```

Figura 3.1

En esta imagen se puede observar cómo, tras iniciar el servidor, hay un único socket UDP activo enlazado a la dirección IP 10.0.0.1 sobre el puerto público 8080. Este es el proceso Acceptor, el cual se encuentra bloqueado a la espera de paquetes SYN entrantes desde cualquier origen (0.0.0.0:*)).

Luego, una vez ingresada la petición de un cliente, el sistema entra en acción. El servidor instancia un nuevo hilo que inmediatamente abre un socket efímero, el cual el sistema operativo le asignó otro puerto (en este caso 35331). En este instante, vemos como coexisten ambos sockets, el del puerto 8080 para escuchar otros clientes, y el 35331 para la transferencia de paquetes con el cliente recién ingresado.

Por último, una vez completado el intercambio de los paquetes de control FIN y ACK, vemos el resultado que el socket efímero fue cerrado y destruido correctamente, liberando sus recursos y evitando fugas de memoria.

3.4. Estructura del paquete:

El paquete que definimos que se enviará a través de la comunicación es el siguiente:

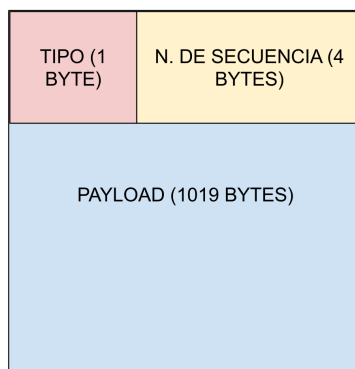


Figura 3.2: Diagrama de la estructura del paquete de aplicación de 1024 bytes.

Como máximo se enviarán 1024 bytes.

- **Tipo (1 Byte):** Especifica el tipo de paquete (Ej: 0 para DATA, 1 para ACK, 2 para SYN, 3 para FIN)
- **Número de Secuencia (4 Bytes):** Un entero sin signo utilizado para controlar el orden y flujo de los paquetes. Gracias a este número, se permite registrar paquetes desordenados o duplicados, que luego se usarán para garantizar la llegada correcta de la información.
- **Payload (1019 Bytes):** Fragmento del archivo de la aplicación.

3.5. Inicio de Comunicación:

Para iniciar una transferencia todo arranca con la conexión del cliente que quiere hacer una acción con el servidor. El cliente inicia mandando un paquete tipo SYN, en el mismo se incluye la metadata como el nombre del archivo o el tamaño, con el objetivo de aprovechar el inicio de transmisión y mandar información relevante. Es importante decir que el mismo se envía mediante la clase protocolo. Esto lo hace mediante el formato de `.ºPERACIÓN|FILENAME|PROTOCOL`". Esto ayuda para que desde el comienzo el servidor ya sepa la intención del cliente (*Upload* o *Download*), el archivo objetivo y el mecanismo control de flujo que debe instanciarse.

El socket general del servidor que es manejado por el *Client Acceptor* que está reiteradamente escuchando por nuevas solicitudes. Cuando recibe un SYN de un cliente el mismo se da cuenta que hay que establecer una nueva conexión por lo que se crea un nuevo *Client Handler* para determinar la transmisión. Como se detalló en la arquitectura general, cada Handler crea y opera sobre su propio socket UDP efímero, independizando la sesión y permitiendo la concurrencia.

Una vez que se crea el *Client Handler*, el mismo responde con un ACK, lo que forma el segundo paso del handshake realizado en la función `recv_syn()` en la clase protocolo.

Cuando el cliente recibe la respuesta del ACK se cambia la dirección del peer. Esto es de suma importancia ya que es aquí donde el cliente deja de comunicarse con el socket general y pasa a interactuar con él con el socket específico del *Client Handler*. Finalizando enviando el último ACK de confirmación y así cerrando el *3-Way-Handshake*.

3.6. Mecanismos de recuperación ante errores

Para garantizar la transferencia confiable sobre UDP y disminuir anomalías de red como la pérdida, el desorden o la corrupción de paquetes, se implementaron dos protocolos en la capa de aplicación: *Stop & Wait* y *Go-Back-N*. Ambos mecanismos basan su estrategia de recuperación en la gestión de *timeouts* propios por la interfaz de los sockets UDP, forzando retransmisiones ante la ausencia de confirmaciones.

3.7. Stop-And-Wait

Stop-And-Wait es un mecanismo de control de flujo que garantiza la entrega íntegra y ordenada de la información. Transmite un único paquete de datos a la vez y bloquea el envío hasta recibir su correspondiente confirmación (ACK).

3.7.1. Manejo de errores

Para manejar las irregularidades en la red, realizamos lo siguiente:

- **Ante Pérdida de paquetes (o ACKs):** Si un paquete se extravía (no llega) o llega corrupto (siendo previamente descartado por el checksum de UDP), el ACK de respuesta nunca llegará. El temporizador del emisor expirará, lo que asume la pérdida del mensaje y dispara su retransmisión inmediata.
- **Ante Paquetes Desordenados o Duplicados:** Cada datagrama incluye un número de secuencia binario (0 o 1) que alterna con cada envío exitoso. Si la red retrasa un paquete y llega fuera de orden (o un duplicado), el receptor detectará que el número de secuencia no coincide con el esperado. Como respuesta, ignora la carga útil para no corromper el archivo, pero reenviará un ACK con el número esperado para destrabar al emisor.

3.7.2. Ventajas y desventajas

Este protocolo tiene como ventaja la garantía de una entrega confiable de la información mediante una implementación simple, sin demasiada complejidad. Sin embargo, se ve contrariada por su baja eficiencia y velocidad de transmisión, dado que el emisor permanece en

estado de espera por mucho tiempo, enviando una baja cantidad de información comparada con la capacidad de la red, desperdiciando la misma.

3.7.3. Flujo de envío (Sender)

La lectura del archivo se procesa en bloques de tamaño fijo (PAYLOAD_SIZE = 1019 BYTES). Por cada bloque:

1. Se construye el paquete de tipo DATA, con su número de secuencia correspondiente y la información contenida en el bloque leído.
2. Se envía el paquete a través del socket UDP.
3. Se espera la recepción del mensaje ACK.
4. Se verifica que el mensaje de ACK contenga el número de secuencia del paquete enviado.
 - a) Si se confirma, se modifica el número de secuencia (1→0, 0→1) y se vuelve al paso 1 con un nuevo bloque.
 - b) Si el número de secuencia es erróneo se ignora.
5. Si se llega al timeout, se retransmite el paquete.

3.7.4. Flujo de recepción (Receiver)

La recepción y procesamiento de los paquetes se realiza de la siguiente forma:

1. Se recibe un paquete de tipo DATA
2. Se verifica que el número de secuencia coincida con el esperado
 - a) Si coincide escribe en el archivo especificado la información recibida y modifica el número de secuencia binario esperado
 - b) Si no coincide ignora el paquete (paquete duplicado o fuera de orden). Se envía un ACK duplicado para solicitar el paquete correcto.
3. Envía un ACK con el número de secuencia actual

3.8. Go-Back-N

El mecanismo *Go-Back-N* es más complejo que *Stop-And-Wait* ya que se permite el envío de más de un paquete consecutivo sin esperar la confirmación de cada uno. Esto se logra mediante una ventana deslizante que define la máxima cantidad de paquetes que pueden haber sin ser confirmados. El emisor puede mandar paquetes hasta llenar la ventana, en el momento que se recibe un ACK la ventana se corre hacia adelante pudiendo transmitir nuevos paquetes. En este protocolo, los ACKs son acumulativos, lo que quiere decir que si se recibe un ACK n , implica que todos los ACKs previos a n se recibieron correctamente. En el caso de *timeouts* (no se recibe un ACK a tiempo) se retransmiten todos los paquetes no confirmados de la ventana.

3.8.1. Manejo de irregularidades en la red

- **Ante pérdida de paquetes:** El emisor mantiene referencias del primer paquete no confirmado (*base*) y el próximo a enviar (*next_seq*). Si se pierde un paquete, los ACKs dejarán de avanzar y el temporizador general se agotará. Ante este *timeout*, el emisor retransmite en ráfaga todos los paquetes pendientes dentro de la ventana activa.
- **Ante paquetes desordenados:** El receptor mantiene una política estricta mediante la variable *expected_seq*. Al carecer de un buffer de reordenamiento, cualquier paquete que llegue adelantado o fuera de orden es descartado inmediatamente. Como respuesta, el receptor transmite un ACK duplicado informando cuál fue el último bloque íntegro recibido, forzando al emisor a retroceder y retransmitir desde ese punto tras el próximo *timeout*.

En síntesis, mientras existan fragmentos y la ventana tenga cupo, el emisor transmite sin pausas. Cuando se ingresa un ACK válido, la ventana se desplaza hacia adelante liberando espacio. Por otro lado del receptor, solo se interactúa con el sistema de archivos cuando el paquete entrante encaja perfectamente en la secuencia de ACKs esperados, garantizando la integridad total del archivo final.

3.9. Manejo de errores de aplicación:

Además de la recuperación de la pérdida de paquetes mediante ACKs, *timeouts* y retransmisiones, el protocolo contempla errores propios de la capa de aplicación. Estos errores se manejan enviando paquetes DATA cuyo payload comienza con formato ERROR|, esto para facilitar el trabajo de reconocimiento del cliente, sabiendo que no tiene que continuar la transmisión de la misma manera. Inicialmente el servidor valida la metadata recibida con el mensaje inicial de SYN. Esta metadata tiene que respetar el formato habitual de OPERACIÓN|FILENAME|PROTOCOL. Si el formato no contiene los tres campos esperados el servidor responde con un mensaje ERROR|BAD_METADATA, así finalizando la transmisión con ese cliente.

También se valida el protocolo solicitado por el cliente. En la implementación se prueba si el campo correspondiente no es ni gbn (*Go-Back-N*) o snw (*Stop-And-Wait*). En el caso que no sea ninguno de esos dos el servidor responde con `ERROR|UNKNOWN_PROTOCOL`, esto ya que el trabajo solo implementa esos protocolos de comunicación.

Finalmente en la operación *download*, el servidor verifica que el archivo solicitado exista en el directorio del almacenamiento. Si el archivo no se encuentra, el servidor responde con `ERROR|FILE_NOT_FOUND` y no inicia la transferencia. De esta manera además de poder identificar fácilmente los errores, gracias al uso de concurrencia un fallo no afecta al funcionamiento general del servidor, sino únicamente al thread asociado a ese cliente, manteniendo el resto de las conexiones activas y ordenadas.

3.10. Fin de Comunicación

El cierre de la comunicación asociado a ambos protocolos se realiza a través del envío de un paquete con encabezado FIN, cuyo objetivo es indicar que no se transmitirá más información de parte del emisor del mensaje.

El emisor envía un paquete FIN con un número de secuencia adjunto. Luego, permanece en espera de un mensaje ACK proveniente del destinatario, que se corresponda con el número de secuencia del paquete de finalización. En caso de una interrupción por *timeout*, es retransmitido hasta un máximo de intentos establecidos en la constante `MAX_FIN_RETRIES`.

Por el lado del receptor, al recibir FIN, responde con el ACK con el número de secuencia del paquete recibido, finaliza la recepción y cierra el archivo.

4. Pruebas

Se realizaron pruebas de transferencia de archivos de 1MB y 5MB, evaluando el comportamiento de las operaciones de subida y descarga. Se comparó el rendimiento de los protocolos *Go-Back-N* y *Stop-And-Wait* en un entorno ideal (sin pérdida) y en una red simulada mediante *Mininet* con un 10% de pérdida de paquetes.

4.0.1. Resultados experimentales

Archivo 1MB:

Operación	Protocolo	Sin Pérdida	Con Pérdida (10 %)
Upload	Go-Back-N	0,25 segundos	6,47 segundos
	Stop-And-Wait	0,31 segundos	12,41 segundos
Download	Go-Back-N	1,26 segundos	8,02 segundos
	Stop-And-Wait	1,37 segundos	13,16 segundos

Tabla 1: Resultados de transferencia para archivo de 1MB.

Archivo 5MB:

Operación	Protocolo	Sin Pérdida	Con Pérdida (10 %)
Upload	Go-Back-N	0,50 segundos	32,42 segundos
	Stop-And-Wait	0,80 segundos	67,37 segundos
Download	Go-Back-N	1,50 segundos	31,68 segundos
	Stop-And-Wait	1,80 segundos	67,41 segundos

Tabla 2: Resultados de transferencia para archivo de 5MB.

4.0.2. Archivo 1MB

```

DEBUG:root:Enviado paquete DATA con seq 1029 a ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1020 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1021 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1022 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1023 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1024 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1025 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1026 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1027 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1028 de ('127.0.0.1', 57464)
DEBUG:root:Recibido ACK para seq 1029 de ('127.0.0.1', 57464)
INFO:root:Enviando FIN con seq 1030 a ('127.0.0.1', 57464)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('127.0.0.1', 57464)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 0.25131702423095703 segundos
python3 upload.py -H 127.0.0.1 -p 8080 -s ../storage/test_1mb.bin -n -r gbn 0.07s user 0.05s system 33% cpu 0.337 total
o facu@Laptop-de-Facundo src %

```

Figura 4.1: GBN (UPLOAD) 1MB sin pérdida de paquetes:

```

DEBUG:root:Recibido paquete DATA con seq 1018 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1019 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1020 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1021 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1022 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1023 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1024 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1025 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1026 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1027 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1028 de ('127.0.0.1', 64245), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 1029 de ('127.0.0.1', 64245), enviado ACK
INFO:root:Recibido FIN con seq 1030 de ('127.0.0.1', 64245)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 1.2671470642089844 segundos
python3 download.py -H 127.0.0.1 -p 8080 -n ../storage/test_1mb.bin -d -r gb 0.06s user 0.05s system 8% cpu 1.352 total
o facu@Laptop-de-Facundo src %

```

Figura 4.2: GBN (DOWNLOAD) 1MB sin pérdida de paquetes:

```

DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 52639) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 52639) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 52639) con seq_num=0, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 52639) con seq_num=0 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 25 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 52639) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 52639) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 0 bytes para enviar
INFO:root:Archivo ../storage/test_1mb.bin enviado completamente, enviando FIN
INFO:root:Enviando FIN con seq 0 a ('127.0.0.1', 52639)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('127.0.0.1', 52639)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 0.3129251003265381 segundos
python3 upload.py -H 127.0.0.1 -p 8080 -s ../storage/test_1mb.bin -n -r snw 0.07s user 0.04s system 28% cpu 0.385 total
o facu@Laptop-de-Facundo src %

```

Figura 4.3: SNW (UPLOAD) 1MB sin pérdida paquetes:

```

INFO:root:DATA recibido de ('127.0.0.1', 61840) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 61840) con seq_num=0
INFO:root:DATA recibido de ('127.0.0.1', 61840) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 61840) con seq_num=1
INFO:root:DATA recibido de ('127.0.0.1', 61840) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 61840) con seq_num=0
INFO:root:DATA recibido de ('127.0.0.1', 61840) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 25 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 61840) con seq_num=1
INFO:root:FIN recibido de ('127.0.0.1', 61840), finalizando recepción de archivo
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 1.3777320384979248 segundos
python3 download.py -H 127.0.0.1 -p 8080 -n ../storage/test_1mb.bin -d -r sn 0.08s user 0.05s system 8% cpu 1.473 total
o facu@Laptop-de-Facundo src %

```

Figura 4.4: SNW (DOWNLOAD) 1MB sin pérdida de paquetes:

```

manu@Manuel: ~/Fiuba/Redes/tp1
DEBUG:root:Enviado paquete DATA con seq 1020 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1011 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1021 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1012 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1022 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1013 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1023 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1014 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1024 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1015 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1025 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1016 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1026 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1017 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1027 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1018 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1019 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1020 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1021 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1022 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1023 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1024 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1025 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1026 a ('10.0.0.1', 46256)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 1027 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1018 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1028 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1019 de ('10.0.0.1', 46256)
DEBUG:root:Enviado paquete DATA con seq 1029 a ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1020 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1021 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1022 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1023 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1024 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1025 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1026 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1027 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1028 de ('10.0.0.1', 46256)
DEBUG:root:Recibido ACK para seq 1029 de ('10.0.0.1', 46256)
INFO:root:Enviando FIN con seq 1030 a ('10.0.0.1', 46256)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('10.0.0.1', 46256)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 6.467403411865234 segundos
mininet>

```

Figura 4.5: GBN (UPLOAD) 1MB con pérdida paquetes:

```

manu@Manuel: ~/Fiuba/Redes/tp1
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=0, esperando ACK...
INFO:root:Timeout esperando ACK de ('10.0.0.1', 49972) para seq_num=0, reintentando...
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=0, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=0 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=0, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=0 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=0, esperando ACK...
INFO:root:Timeout esperando ACK de ('10.0.0.1', 49972) para seq_num=1, reintentando...
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=0, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=0 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 25 bytes para enviar
DEBUG:root:DATA enviado a ('10.0.0.1', 49972) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('10.0.0.1', 49972) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 0 bytes para enviar
INFO:root:Archivo storage/lnagen_1mb.png enviado completamente, enviando FIN
INFO:root:Enviando FIN con seq 0 a ('10.0.0.1', 49972)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('10.0.0.1', 49972)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 13.16150188446045 segundos
mininet>

```

Figura 4.6: SNW (UPLOAD) 1MB con pérdida paquetes:


```
manu@Manuel: ~/Fiuba/Redes/tp1
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=0
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=0
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=0
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=0
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=0
INFO:root:DATA recibido de ('10.0.0.1', 42152) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 25 bytes
DEBUG:root:ACK enviado a ('10.0.0.1', 42152) con seq_num=1
INFO:root:FIN recibido de ('10.0.0.1', 42152), finalizando recepción de archivo
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 12.411959886550903 segundos
mininet>
```

Figura 4.7: SNW (DOWNLOAD) 1MB con pérdida paquetes:

4.0.3. Archivo 5MB:

```
DEBUG:root:Recibido paquete DATA con seq 5134 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5135 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5136 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5137 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5138 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5139 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5140 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5141 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5142 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5143 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5144 de ('127.0.0.1', 56327), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5145 de ('127.0.0.1', 56327), enviado ACK
INFO:root:Recibido FIN con seq 5146 de ('127.0.0.1', 56327)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 1.5228910446166992 segundos
python3 download.py -H 127.0.0.1 -p 8080 -n ../storage/test_5mb.bin -d -r gb 0.15s user 0.16s system 19% cpu 1.595 total
o facu@Laptop-de-Facundo src %
```

Figura 4.8: GBN (Download) 5MB sin pérdida de paquetes


```

DEBUG:root:Enviado paquete DATA con seq 5145 a ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5136 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5137 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5138 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5139 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5140 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5141 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5142 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5143 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5144 de ('127.0.0.1', 53552)
DEBUG:root:Recibido ACK para seq 5145 de ('127.0.0.1', 53552)
INFO:root:Enviando FIN con seq 5146 a ('127.0.0.1', 53552)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('127.0.0.1', 53552)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 0.5030021667480469 segundos
python3 upload.py -H 127.0.0.1 -p 8080 -s ../storage/test_5mb.bin -n -r gbn 0.17s user 0.17s system 56% cpu 0.587 total
o facu@Laptop-de-Facundo src %

```

Figura 4.9: GBN (UPLOAD) 5MB sin pérdida de paquetes:

```

DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 51681) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 51681) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 1019 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 51681) con seq_num=0, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 51681) con seq_num=0 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 125 bytes para enviar
DEBUG:root:DATA enviado a ('127.0.0.1', 51681) con seq_num=1, esperando ACK...
DEBUG:root:ACK recibido de ('127.0.0.1', 51681) con seq_num=1 correcto, continuando con el siguiente chunk
DEBUG:root:Leído del archivo 0 bytes para enviar
INFO:root:Archivo ../storage/test_5mb.bin enviado completamente, enviando FIN
INFO:root:Enviando FIN con seq 0 a ('127.0.0.1', 51681)
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('127.0.0.1', 51681)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 0.8308773040771484 segundos
python3 upload.py -H 127.0.0.1 -p 8080 -s ../storage/test_5mb.bin -n -r snw 0.22s user 0.13s system 37% cpu 0.911 total
o facu@Laptop-de-Facundo src %

```

Figura 4.10: SNW (UPLOAD) 5MB sin pérdida de paquetes:

```

INFO:root:DATA recibido de ('127.0.0.1', 55087) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 55087) con seq_num=0
INFO:root:DATA recibido de ('127.0.0.1', 55087) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 55087) con seq_num=1
INFO:root:DATA recibido de ('127.0.0.1', 55087) con seq_num=0, escribiendo en archivo
DEBUG:root:Payload del DATA: 1019 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 55087) con seq_num=0
INFO:root:DATA recibido de ('127.0.0.1', 55087) con seq_num=1, escribiendo en archivo
DEBUG:root:Payload del DATA: 125 bytes
DEBUG:root:ACK enviado a ('127.0.0.1', 55087) con seq_num=1
INFO:root:FIN recibido de ('127.0.0.1', 55087), finalizando recepción de archivo
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 1.8402469158172607 segundos
python3 download.py -H 127.0.0.1 -p 8080 -n ../storage/test_5mb.bin -d -r sn 0.22s user 0.13s system 18% cpu 1.916 total
o facu@Laptop-de-Facundo src %

```

Figura 4.11: SNW (DOWNLOAD) 5MB sin pérdida de paquetes:

```

manu@Manuel: ~/Fiuba/Redes/tp1
DEBUG:root:Enviado paquete DATA con seq 5136 a ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5137 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5128 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5138 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5129 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5139 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5130 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5140 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5131 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5141 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5132 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5142 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5133 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5134 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5135 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5136 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5137 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5138 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5139 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5140 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5141 a ('10.0.0.1', 33180)
DEBUG:root:Timeout: Reenviado paquete DATA con seq 5142 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5133 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5143 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5134 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5144 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5135 de ('10.0.0.1', 33180)
DEBUG:root:Enviado paquete DATA con seq 5145 a ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5136 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5137 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5138 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5139 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5140 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5141 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5142 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5143 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5144 de ('10.0.0.1', 33180)
DEBUG:root:Recibido ACK para seq 5145 de ('10.0.0.1', 33180)
INFO:root:Enviando FIN con seq 5146 a ('10.0.0.1', 33180)
DEBUG:root:Timeout esperando ACK de FIN de ('10.0.0.1', 33180), reintentando FIN...
INFO:root:Recibiendo ACK de FIN. Cierre de conexión confirmado por ('10.0.0.1', 33180)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 32.42444586753845 segundos
mininet>

```

Figura 4.12: GBN (UPLOAD) 5MB con pérdida paquetes:

```

manu@Manuel: ~/Fiuba/Redes/tp1
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5138 de ('10.0.0.1', 58459), esperado 5130. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5129 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA con seq 5130 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5131 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5132 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5133 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5134 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5135 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5136 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5137 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5138 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5140 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5141 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5142 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5143 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5144 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5145 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA fuera de orden con seq 5146 de ('10.0.0.1', 58459), esperado 5139. Reenviando ACK del último paquete correcto.
DEBUG:root:Reenviado ACK para seq 5138 a ('10.0.0.1', 58459)
DEBUG:root:Recibido paquete DATA con seq 5139 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5140 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5141 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5142 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5143 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5144 de ('10.0.0.1', 58459), enviado ACK
DEBUG:root:Recibido paquete DATA con seq 5145 de ('10.0.0.1', 58459), enviado ACK
INFO:root:Recibido FIN con seq 5146 de ('10.0.0.1', 58459)
DEBUG:root:Thread Sender detenido
DEBUG:root:Thread Receiver detenido
Tiempo total de transferencia: 31.67723298072815 segundos
mininet>

```

Figura 4.13: GBN (DOWNLOAD) 5MB con pérdida paquetes:

Figura 4.14: SNW (UPLOAD) 5MB con pérdida paquetes:

Figura 4.15: SNW (DOWNLOAD) 5MB con pérdida paquetes:

4.1. Análisis de Throughput (rendimiento)

Throughput (Bytes que se transfieren / tiempo en transferir) Archivo 1MB:

Archivo 1MB:

Protocolo	Go-Back-N	Stop and Wait
Throughput (Upload)	4 MB/s	3,23 MB/s
Throughput (10 % pérdida)	0,15 MB/s	0,08 MB/s
Throughput (Download)	0,79 MB/s	0,73 MB/s
Throughput (10 % pérdida)	0,12 MB/s	0,08 MB/s

Tabla 3: Throughput para archivo de 1MB.

Archivo 5MB:

Protocolo	Go-Back-N	Stop and Wait
Throughput (Upload)	10 MB/s	6,25 MB/s
Throughput (10 % pérdida)	0,15 MB/s	0,07 MB/s
Throughput (Download)	3,33 MB/s	2,78 MB/s
Throughput (10 % pérdida)	0,16 MB/s	0,07 MB/s

Tabla 4: Throughput para archivo de 5MB.

Como se puede observar en las tablas de rendimiento, los datos empíricos confirman la hipótesis inicial: *Go-Back-N* presenta una superioridad absoluta frente a *Stop-And-Wait* en todos los escenarios.

En condiciones ideales (0% de pérdida), *Go-Back-N* logra un throughput de subida hasta 10MB/s, superando ampliamente los 6,25 MB/s de *Stop-And-Wait*. Esta diferencia se vuelve aún más crítica en escenarios de congestión (10% pérdida), donde el mecanismo de ventana deslizante *Go-Back-N* logra recuperar la información en la mitad que le toma a *Stop-And-Wait* (32 segundos vs 67 segundos para 5MB, casi el doble del tiempo), demostrando un uso mucho más eficiente del canal de red ante irregularidades.

4.2. Análisis de Protocolos con Wireshark

4.2.1. Pérdida de paquetes con Stop and Wait

344	47.217524607	10.0.0.2	10.0.0.1	UDP	1068	42832 → 52481	Len=1024
345	47.268057394	10.0.0.2	10.0.0.1	UDP	1068	42832 → 52481	Len=1024
346	47.319531694	10.0.0.2	10.0.0.1	UDP	1068	42832 → 52481	Len=1024
347	47.319550302	10.0.0.2	10.0.0.1	UDP	1068	42832 → 52481	Len=1024

Figura 4.16

- ▶ Frame 344: 1068 bytes on wire (854)
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4, Src:
- ▶ User Datagram Protocol, Src Port:
- ▼ Data (1024 bytes)
 - Data [truncated]: 000000000082c
 - [Length: 1024]

- ▶ Frame 345: 1068 bytes on wire (854)
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4, Src:
- ▶ User Datagram Protocol, Src Port:
- ▼ Data (1024 bytes)
 - Data [truncated]: 000000000082c
 - [Length: 1024]

Figura 4.17

- ▶ Frame 346: 1068 bytes on wire (854)
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4, Src:
- ▶ User Datagram Protocol, Src Port:
- ▼ Data (1024 bytes)
 - Data [truncated]: 000000000082c
 - [Length: 1024]

- ▶ Frame 347: 1068 bytes on wire (854)
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4, Src:
- ▶ User Datagram Protocol, Src Port:
- ▼ Data (1024 bytes)
 - Data [truncated]: 000000000082c
 - [Length: 1024]

Figura 4.18

- ▶ Frame 348: 49 bytes on wire
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4,
- ▶ User Datagram Protocol, Src
- ▼ Data (5 bytes)
 - Data: 010000000
 - [Length: 5]

- ▶ Frame 350: 1068 bytes on wire (854)
- ▶ Linux cooked capture v1
- ▶ Internet Protocol Version 4, Src:
- ▶ User Datagram Protocol, Src Port:
- ▼ Data (1024 bytes)
 - Data [truncated]: 0000000001658
 - [Length: 1024]

Figura 4.19

En las siguientes imágenes se muestra en práctica el protocolo de recuperación ante errores *Stop and Wait*. La información provista por *Wireshark* muestra el envío y la recepción de

paquetes de parte de ambos nodos presentes en *Mininet*. Ante el 10% de probabilidades de pérdida de paquetes, la data enviada en el frame 344 se ve descartada en su viaje por la red. Luego de transcurrido el intervalo de *timeout*, el paquete es retransmitido, lo que se puede ver en el frame 345, donde el paquete contiene el mismo número de secuencia 0. Nuevamente, se produce la pérdida de este en los nodos intermedios, por lo tanto al finalizar el periodo de *timeout* es enviado en el frame 346 y podemos darnos cuenta que es recepcionado por el destinatario en el frame 347. Posteriormente, se envía el ACK que indica la recepción del respectivo paquete en el frame 348, por lo tanto, en el frame 350 se envían nuevos datos bajo el número de secuencia 1.

4.3. Pérdida de paquetes y manejo de ventana de envío bajo Go-Back-N

26	7.115249627	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
27	7.115530001	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
28	7.115581279	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
29	7.115614261	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
30	7.115643644	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
31	7.115654393	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
32	7.115681768	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
33	7.116355100	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
34	7.116369279	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
35	7.116386678	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
36	7.116578103	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
37	7.116881464	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
38	7.116895930	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
39	7.116906267	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
40	7.116915893	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
41	7.116926517	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
42	7.116936469	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
43	7.117272569	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
44	7.117293479	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
45	7.117306052	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
46	7.117317443	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
47	7.117421456	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
48	7.117436800	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
49	7.117447283	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
50	7.117457319	10.0.0.1	10.0.0.2	UDP	47 49872 → 57183 Len=5
51	7.170890249	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
52	7.170970247	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
53	7.170988247	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
54	7.171002177	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
55	7.171011446	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
56	7.171021992	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
57	7.171032859	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
58	7.171043582	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
59	7.171054435	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024
60	7.171065188	10.0.0.2	10.0.0.1	UDP	1066 57183 → 49872 Len=1024

Figura 4.20

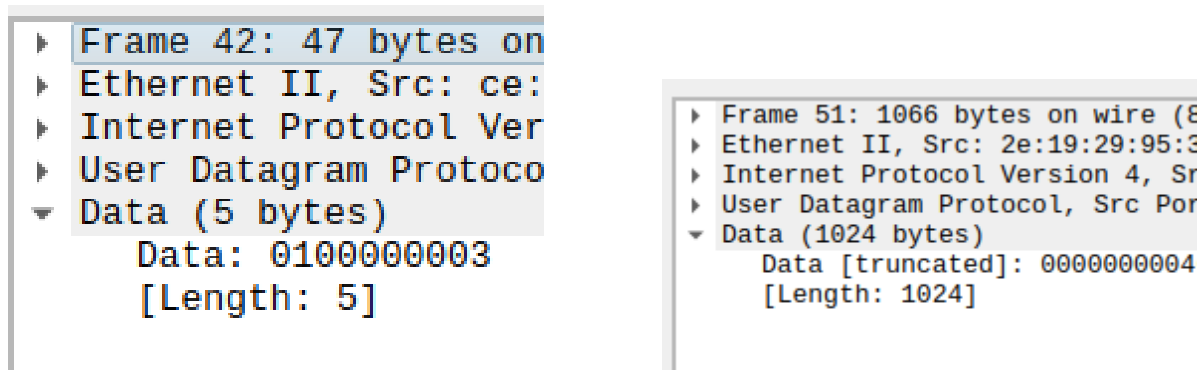


Figura 4.21

En estas capturas obtenidas se puede analizar el manejo de la ventana de envío utilizada en el protocolo *Go-Back-N*. En primer lugar, se envían todos los paquetes presentes en la ventana de tamaño 10, cada uno con sus respectivos números de secuencia. Al ser recepcionados, se puede comenzar a ver la llegada de una serie de paquetes ACK provenientes del receptor. Se reciben 7 ACK's, sin embargo, se indica que se recibió hasta el paquete con número de secuencia 3, tomando aquel con mayor número de aseveración de recepción, ubicado en el frame 42 de *Wireshark*. Por esta razón, la ventana se mueve 4 lugares al asegurarse la llegada de los paquetes 0, 1, 2 y 3, enviando los siguientes datos. La llegada del resto se pudo ver afectada por congestión, desordenamiento o pérdida de paquetes.

Los siguientes números de ACK recibidos por el emisor permanecen siendo 3. Esto nos da un indicio de que aquel que contenía el número de secuencia 4 fue descartado o corrompido. Cuando se llega a la finalización del intervalo de *timeout*, se asume la pérdida del paquete, por lo que el emisor envía nuevamente la totalidad de la ventana, comenzando desde el paquete 4 (frame 51 en adelante).

5. Preguntas teóricas

1. Describa la arquitectura Cliente-Servidor.
2. ¿Cuál es la función de un protocolo de capa de aplicación?
3. Detalle el protocolo de aplicación desarrollado en este trabajo.
4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?
5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.
6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

5.1. Arquitectura Cliente-Servidor

La Arquitectura Cliente-Servidor es un modelo de comunicación basado en la interacción de dos roles bien definidos: un cliente que inicia una comunicación enviando una petición de lo que necesita y un servidor que mantiene la escucha para procesar lo pedido y devolver la respuesta correspondiente.

En el contexto más específico de nuestro trabajo práctico decimos que el cliente solicita operaciones de transferencias de archivos (*Upload* y *Download*) y el servidor procesa el pedido y envía los archivos correspondientes. Profundizando en la implementación vemos que la comunicación sucede mediante sockets en la red usando el protocolo UDP. La información se divide en distintos paquetes usando protocolos de transferencia como *Stop-And-Wait*, entre otros. Además, el servidor se diseñó de manera concurrente con la utilización de un *client-handler* para dividir cada conexión siendo independientes en base a threads.

5.2. ¿Cuál es la función de un protocolo de capa de aplicación?

La función de un protocolo de capa de aplicación es definir las bases (reglas y formatos) para que dos hosts en aplicaciones distintas puedan comunicarse dentro de la red.

Entre las funciones principales que tiene que cumplir la capa, podemos destacar: la definición de la estructura de los mensajes que se intercambian; el orden que deben enviarse, lo cual es importante para evitar malas interpretaciones; asegurarse que la información sea comprendida por ambos extremos, esto teniendo de por medio un proceso de decodificación; y formalizar la manera de interactuar por primera vez para que no hayan errores, entre otros.

5.3. Detalle del protocolo de aplicación desarrollado

El protocolo de aplicación desarrollado es binario y orientado a la transferencia confiable sobre UDP. Consta de una cabecera fija de 5 bytes (1 byte para tipo de mensaje y 4 para el número de secuencia) emparejada con un payload máximo de 1019 bytes.

Utiliza un *3-way-handshake* adaptado para acordar la operación (DOWNLOAD o UPLOAD), el nombre del archivo, tipo de protocolo a usarse (*Go-Back-N* o *Stop and Wait*) y delegar la conexión a puertos efímeros dinámicos.

Para la transferencia, se aplica el mecanismo deseado por el cliente. Si se utiliza *Stop-and-Wait*, el protocolo enviará un paquete y bloqueará su hilo Sender hasta que el Receiver obtenga la confirmación. En cambio, si se utiliza *Go-Back-N*, el hilo Sender enviará ráfagas de paquetes limitados por una ventana estática, mientras el hilo Receiver procesa ACKs acumulativos.

Y finalmente, la comunicación finaliza mediante el envío de un paquete FIN, exigiendo su correspondiente ACK de validación (con un sistema de reintentos máximos) para garantizar la

liberación correcta de recursos.

5.4. TCP vs UDP

La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. Cada protocolo tiene características distintas dependiendo del contexto específico de la arquitectura cliente-servidor que se utilice. Cada uno debería, dependiendo del trabajo, pensar que protocolo le conviene usar en ese momento.

Empezando por TCP podemos decir que los servicios principales que ofrecen son la confiabilidad en la entrega de datos, asegurando que los paquetes enviados lleguen correctamente y en orden, aun siendo más lenta la comunicación. Además implementa mecanismos de control de flujo y de congestión pensando en no tener la red inundada de paquetes, haciendo que la comunicación sea más escalable y estable. También establece las nuevas conexiones mediante la ayuda de un handshake, permitiendo una comunicación más controlada entre cliente y servidor, entre otros.

Por otro lado encontramos UDP, el mismo es un protocolo de capa de transporte más sencillo que TCP, lo que lo hace más rápido pero menos confiable. Una de las cosas que lo diferencia es que no establece conexiones previamente, perdiendo confiabilidad. Tampoco cuenta con mecanismos ni de control de congestión ni para el orden de los paquetes.

En el caso de TCP se debería aplicar cuando la aplicación requiere confiabilidad y correcta entrega de datos, aunque no se priorice la velocidad. Como ejemplos encontramos las transacciones bancarias o el servicio de mail. En cambio UDP se adecua mejor en situaciones donde se requiera rapidez por sobre confiabilidad, como en el caso de los juegos online donde cada segundo cuenta y las pérdidas de paquetes son tolerables.

5.5. Control de congestión

El protocolo desarrollado NO cuenta con un mecanismo de control de congestión, sino que se apega a la definición teórica de *Go-Back-N*. Nuestra implementación limita la cantidad de paquetes en vuelo con una ventana estática para no desbordar al receptor. El emisor no adapta su tasa de inyección de paquetes a la saturación de los routers. En caso de necesitar enviar la congestión en la red, se requerirá implementar una ventana de congestión dinámica; aplicando un algoritmo como Tahoe o Reno, con fases de Slow Start o Congestion Avoidance.

5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo?

A partir de las pruebas realizadas, *Go-Back-N* envía archivos en una cantidad de tiempo significativamente menor. Por ejemplo, en las simulaciones con un 10% de pérdida de paquetes, GBN logró descargar el archivo en aproximadamente 32 segundos, frente a los más de 67 segundos que demoró SNW (representando más del doble de tiempo).

GBN es consistentemente más rápido porque mantiene N paquetes en tránsito llenando el tubo (ya que funciona como un pipeline) y aprovechando el ancho de banda disponible. Por otro lado, SNW desperdicia el tiempo de ida y vuelta (RTT) ya que el emisor queda inactivo y bloqueado por la espera de confirmación de cada paquete individual enviado.

6. Dificultades encontradas

Una de las principales dificultades encontradas fue la implementación del protocolo de aplicación sobre UDP. Como nombramos en la sección de preguntas a responder, UDP no implementa mecanismos esenciales como control de flujo, formato de paquetes, timeouts, etc. Esto llevó a que nosotros tuviéramos que implementarlos de base lo que fue muy complejo, más específico manejar duplicados o retransmisiones innecesarias.

Otra de las dificultades más grandes que encontramos fue la implementación de concurrencia. El hecho de manejar las peticiones de distintos clientes de manera simultánea fue difícil principalmente en la arquitectura general del servidor. La dificultad estuvo en organizar a los clientes para que cada uno tuviera su propio thread y no haya interferencias entre las demás conexiones, más en específico entre el manejo de sockets y el envío de paquetes. Para esto tuvimos que crear una separación entre un Acceptor, que recibía solicitudes, y varios *Client-Handlers*, que manejaban a cada cliente independientemente, haciendo así todo más ordenado.

Unido a este problema de concurrencia, nos enfrentamos a la aparición de hilos zombies. Al interrumpir el servidor enviando la señal SIGINT (con Ctrl + C), el proceso no terminaba su ejecución y la terminal quedaba bloqueada. Esto ocurría porque los hilos secundarios quedaban atrapados en llamadas bloqueantes al sistema. Por lo que, se debió implementar un mecanismo de apagado que cierre los sockets forzosamente desde el hilo principal para liberar sus recursos.

Finalmente nos encontramos con la dificultad del cierre de la comunicación. Para finalizar el protocolo surgieron distintas alternativas de implementación. Inicialmente cuando el emisor recibía un ACK luego de enviar el paquete FIN se cerraba la transmisión. Posteriormente notamos que esta solución podría traer problemas en un futuro, ya que existía la posibilidad de que el ACK recibido posterior a FIN no correspondiera realmente al FIN sino a un paquete retrasado o duplicado. Esto podría producir cierres incorrectos y generar inconsistencias en la

finalización de la comunicación. Para resolverlo se decidió mantener un paquete específico de tipo FIN específicamente para el cierre pero además validar que el ACK recibido corresponda exactamente al número de secuencia de ese FIN. De esta forma logrando un cierre más robusto y terminar con confirmaciones incorrectas de cierre de comunicación.

7. Conclusión

A lo largo del trabajo logramos implementar una aplicación de transferencia confiable de archivos sobre UDP, utilizando una arquitectura cliente-servidor concurrente. La evolución del trabajo nos permitió aplicar en la práctica conceptos vistos en la materia como el uso de sockets, protocolo de transporte, diseño de aplicación de red y manejo de múltiples clientes en simultáneo.

Uno de los principales conceptos fue comprender la complejidad de construir confiabilidad sobre UDP. Al entenderse de un protocolo no orientado a conexión como UDP, sin garantías de orden o de entrega, fue necesario implementar en la capa de aplicación mecanismos como handshake, número de secuencia (posición el paquete enviado), ACKs, retransmisiones, *timeouts* y cierre de aplicación. Esto permitió entender a mayor profundidad qué problemas resuelve UDP y que implica tener que resolverlos manualmente.

Además de los protocolos *Stop-And-Wait* y *Go-Back-N*, uno de los aspectos más importantes del trabajo fue el diseño del servidor concurrente. La separación entre *Client Acceptor* y múltiples *Client Handlers* permitió manejar distintos clientes en paralelo sin que una transferencia bloquee al resto. A su vez, la generalización del envío y recepción mediante el uso de los hilos sender y receiver permitió independizar la lógica del protocolo de las operaciones bloqueantes del socket. De esta forma, tanto el cliente como el *Client Handler* pueden reutilizar funciones `send_file` y `recv_file` de forma ordenada, delegando la comunicación a bajo nivel a estos hilos internos, simplificando el flujo general de transferencia.

En conclusión, este trabajo nos permitió no solo implementar una solución funcional, sino que también entender desafíos reales de diseñar protocolos confiables implementando lo visto en las clases teóricas en un trabajo concreto.